

PO SZKOLENIU

Dalsze kroki

Ścieżka rozwoju, certyfikacja i funkcje warte poznania

Spis treści

CZĘŚĆ 1	Ścieżka rozwoju	3
CZĘŚĆ 2	Certyfikacja	3
CZĘŚĆ 3	Auto Loader	4
CZĘŚĆ 4	Lakeflow Pipelines	5
CZĘŚĆ 5	Unity Catalog w szczegółach	7
CZĘŚĆ 6	Structured Streaming	8
CZĘŚĆ 7	Wydajność	9
CZĘŚĆ 8	MLflow	10
CZĘŚĆ 9	Zaawansowana orkestracja	11
CZĘŚĆ 10	Integracje	12

Szkolenie Databricks Fundamental wprowadza w platformę, Sparka, Delta Lake, orkestrację i Unity Catalog. Ten przewodnik podpowiada, czego uczyć się dalej: jak przejść przez kolejne szkolenia i certyfikację oraz które funkcje platformy warto poznać po opanowaniu podstaw.

CZĘŚĆ 1

Ścieżka rozwoju

Etap	Szkolenie lub certyfikat	Zakres
Podstawy	Databricks Fundamental	Platforma, Spark, Delta Lake, Lakeflow Jobs, Unity Catalog
Kolejne szkolenie	Databricks Explorer	Następne szkolenie na ścieżce Altkom Akademii
Certyfikacja	Databricks Certified Data Engineer Associate	Codzienna praca inżyniera danych w Databricks
Poziom zaawansowany	Databricks Certified Data Engineer Professional	Pipeline'y produkcyjne, wydajność i bezpieczeństwo
Specjalizacja	Uczenie maszynowe, analityka SQL, administracja platformą	Ścieżki dla konkretnych ról

Databricks Academy (databricks.com/learn/training) oferuje bezpłatne kursy do samodzielnej nauki i egzaminy próbne dla każdego etapu.

CZĘŚĆ 2

Certyfikacja

Data Engineer Associate

Naturalny kolejny certyfikat po tym szkoleniu. Obejmuje Delta Lake, Sparka, platformę Databricks, architekturę medalionową i podstawy ETL.

Element	Szczegóły
Forma	45 pytań jednokrotnego wyboru, 90 minut
Opłata	200 USD
Ważność	Dwa lata, potem recertyfikacja
Zalecane doświadczenie	To szkolenie i około trzech miesięcy praktyki

Associate Developer for Apache Spark

Ten certyfikat koncentruje się na DataFrame API i Spark SQL, a nie na platformie Databricks, dlatego najbardziej przyda się osobom, które pracują ze Sparkiem także poza Databricks. Aktualną formę egzaminu i opłatę sprawdź w przewodniku egzaminacyjnym.

Data Engineer Professional

Obejmuje zaawansowane zagadnienia Delta Lake, Lakeflow Pipelines, Unity Catalog, strojenie wydajności i bezpieczeństwo. Wcześniej zalecane jest uzyskanie certyfikatu Associate.

CZĘŚĆ 3

Auto Loader

Auto Loader, czyli źródło `cloudFiles`, pobiera pliki z magazynu w chmurze przyrostowo. Zapamiętuje pliki już przetworzone, więc przy każdym uruchomieniu czyta tylko nowe, zamiast listować i skanować cały katalog. Obsługuje ADLS Gen2, Amazon S3 i Google Cloud Storage, sam wykrywa i rozwija schemat, a w Lakeflow Pipelines jest standardowym sposobem pobierania plików.

```
# Read new files incrementally
df = (spark.readStream
      .format("cloudFiles")
      .option("cloudFiles.format", "json")
      .option("cloudFiles.schemaLocation", "/Volumes/main/bronze/checkpoints/orders_schema")
      .load("/Volumes/main/landing/raw/orders/"))

# Append them to a Bronze table and stop once every available file is processed
(df.writeStream
  .option("checkpointLocation", "/Volumes/main/bronze/checkpoints/orders")
  .trigger(availableNow=True)
  .toTable("main.bronze.orders"))
```

Opcja	Zastosowanie
<code>cloudFiles.format</code>	Format plików źródłowych, np. json, csv, parquet lub avro
<code>cloudFiles.schemaLocation</code>	Miejsce zapisu wykrytego schematu; użyj ścieżki wolumenu Unity Catalog
<code>cloudFiles.inferColumnTypes</code>	Wykrywanie typów kolumn w JSON i CSV zamiast wczytywania wszystkich kolumn jako tekst
<code>cloudFiles.schemaEvolutionMode</code>	Sposób obsługi nowych kolumn pojawiających się w źródle
<code>cloudFiles.maxFilesPerTrigger</code>	Maksymalna liczba plików przetwarzanych w jednej mikropartii

Dokumentacja: docs.databricks.com/en/ingestion/cloud-object-storage/auto-loader/index.html

CZĘŚĆ 4

Lakeflow Pipelines

Lakeflow Pipelines, dawniej Delta Live Tables, to deklaratywny framework do produkcyjnego ETL. Tabele deklarujesz w Pythonie lub SQL, a Databricks ustala kolejność wykonania, przetwarza dane przyrostowo, stosuje reguły jakości danych i zapisuje metryki każdej aktualizacji.

Tworzenie pipeline'u

1. Na pasku bocznym workspace otwórz **Jobs & Pipelines** i utwórz pipeline ETL.
2. Nadaj pipeline'owi nazwę, na przykład `retailhub_pipeline`, i dodaj pliki źródłowe.
3. Jako miejsce docelowe ustaw Unity Catalog i wybierz katalog oraz schemat docelowy.
4. Wybierz zasoby serverless.

Kod pipeline'u

API Pythona importuje się poleceniem `from pyspark import pipelines as dp`. Streaming tables deklaruje się dekoratorem `@dp.table` lub funkcją `dp.create_streaming_table()`, a materialized views dekoratorem `@dp.materialized_view`.

```

from pyspark import pipelines as dp
from pyspark.sql.functions import col, sum as sum_

# Bronze: raw orders, appended as new files arrive
dp.create_streaming_table("orders_bronze", comment="Raw orders, append-only")

@dp.append_flow(target="orders_bronze")
def orders_bronze_flow():
    return (spark.readStream
            .format("cloudFiles")
            .option("cloudFiles.format", "json")
            .option("cloudFiles.inferColumnTypes", "true")
            .load("/Volumes/main/landing/raw/orders/"))

# Silver: rows that break a rule are dropped and counted
dp.create_streaming_table(
    "orders_silver",
    comment="Validated orders",
    expect_all_or_drop={
        "valid_order_id": "order_id IS NOT NULL",
        "positive_amount": "total_amount > 0",
    },
)

@dp.append_flow(target="orders_silver")
def orders_silver_flow():
    return (spark.readStream.table("orders_bronze")
            .withColumn("order_date", col("order_datetime").cast("date"))
            .drop("order_datetime"))

# Gold: a materialized view refreshed on every pipeline update
@dp.materialized_view(name="revenue_by_day", comment="Daily revenue by payment method")
def revenue_by_day():
    return (spark.read.table("orders_silver")
            .groupBy("order_date", "payment_method")
            .agg(sum_("total_amount").alias("daily_revenue")))

```

Element	Zastosowanie
<code>dp.create_streaming_table()</code>	Deklaruje streaming table, do której zapisuje jeden lub więcej przepływów
<code>@dp.append_flow(target=...)</code>	Definiuje zapytanie dopisujące dane do streaming table
<code>@dp.materialized_view</code>	Deklaruje materialized view zbudowany z zapytania wsadowego
<code>expect_all_or_drop</code>	Odrzuca wiersze łamiące regułę i zapisuje, ile ich odrzucono
<code>expect_all</code>	Zachowuje wszystkie wiersze i zapisuje naruszenia reguł jako metryki
<code>expect_all_or_fail</code>	Zatrzymuje aktualizację, gdy którykolwiek wiersz łamie regułę
<code>spark.readStream.table()</code>	Czyta inną tabelę pipeline'u przyrostowo
<code>spark.read.table()</code>	Czyta inną tabelę pipeline'u w całości

Uruchamianie i monitorowanie

Aktualizacja może odświeżać tabele przyrostowo, przetwarzając tylko nowe dane, albo wykonać pełne odświeżenie, które je przebudowuje. Pipeline'y można harmonogramować tak samo jak joby. W trakcie aktualizacji graf pipeline'u pokazuje stan każdej tabeli, a po jej zakończeniu widok jakości danych pokazuje dla każdej reguły, ile wierszy przeszło, a ile odrzucono.

Wiersze usunięte przez `expect_all_or_drop` nie są nigdzie zachowywane. Jeśli chcesz je analizować, zapisz je samodzielnie do osobnej tabeli, stosując odwrotność reguły.

Pipeline'y a joby z notebookami

	Notebooki w Lakeflow Jobs	Lakeflow Pipelines
Kolejność wykonania	Zależności zadań definiowane ręcznie	Wynika z odwołań między tabelami
Jakość danych	Własne sprawdzenia w kodzie	Deklarowane oczekiwania z wbudowanymi metrykami
Przetwarzanie przyrostowe	Implementowane ręcznie	Zapewniane przez framework
Po awarii	Repair run zadań zakończonych błędem	Automatyczne ponowienia
Monitorowanie	Logi i wyniki zadań	Graf pipeline'u i metryki jakości danych

Dokumentacja: docs.databricks.com/en/ldp/

CZĘŚĆ 5

Unity Catalog w szczegółach

Na szkoleniu Unity Catalog służył do obsługi trzypoziomowej przestrzeni nazw i podstawowych uprawnień. Oferuje on także precyzyjniejsze mechanizmy kontroli.

Filtry wierszy

Filtr wierszy to funkcja SQL, która zwraca prawdę dla wierszy, które użytkownik może zobaczyć. Przypina się ją do tabeli, a Databricks oblicza ją przy każdym zapytaniu.

```
CREATE FUNCTION main.security.region_filter(region STRING)
RETURN is_account_group_member('sales_admins') OR region = 'EMEA';

ALTER TABLE main.sales.transactions
SET ROW FILTER main.security.region_filter ON (region);
```

Maski kolumn

Maska kolumny to funkcja SQL, która zwraca wartość pokazywaną użytkownikowi zamiast wartości zapisanej.

```
CREATE FUNCTION main.security.email_mask(email STRING)
RETURN CASE
    WHEN is_account_group_member('data_admins') THEN email
    ELSE regexp_replace(email, '^[^@]+', '***')
END;

ALTER TABLE main.retail.customers
ALTER COLUMN email SET MASK main.security.email_mask;
```

Lineage

Unity Catalog bez żadnej konfiguracji zapisuje lineage na poziomie tabel i kolumn dla zapytań, notebooków i jobów. Widać go na karcie Lineage tabeli w Catalog Explorer, a dane można też odpytać w system.access.table_lineage.

Tagi

```
ALTER TABLE main.retail.customers SET TAGS ('pii' = 'true', 'domain' = 'customer');

ALTER TABLE main.retail.customers ALTER COLUMN email SET TAGS ('sensitivity' = 'high');
```

Delta Sharing

Delta Sharing daje innym organizacjom dostęp tylko do odczytu do aktualnych tabel bez kopiowania danych. Odbiorcy nie potrzebują workspace'u Databricks; mogą czytać udostępnione dane otwartymi klientami dla Pythona, Sparka, pandas lub Power BI.

Dokumentacja: docs.databricks.com/en/data-governance/unity-catalog/index.html

CZĘŚĆ 6

Structured Streaming

Auto Loader działa na Structured Streaming, które przetwarza nieograniczony strumień danych jako serię małych partii.

```

from pyspark.sql.functions import from_json, col, window, sum as sum_
from pyspark.sql.types import StructType, StructField, StringType, DoubleType, TimestampType

schema = StructType([
    StructField("order_id", StringType()),
    StructField("payment_method", StringType()),
    StructField("total_amount", DoubleType()),
    StructField("event_time", TimestampType()),
])

orders = (spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "broker:9092")
    .option("subscribe", "orders")
    .load()
    .select(from_json(col("value").cast("string"), schema).alias("o"))
    .select("o.*"))

revenue = (orders
    .withWatermark("event_time", "10 minutes")
    .groupBy(window("event_time", "5 minutes"), "payment_method")
    .agg(sum_("total_amount").alias("revenue")))

(revenue.writeStream
    .outputMode("append")
    .option("checkpointLocation", "/Volumes/main/gold/checkpoints/revenue_5min")
    .toTable("main.gold.revenue_5min"))

```

Wyzwalacz	Działanie
processingTime="1 minute"	Uruchamia mikropartię w stałych odstępach czasu
availableNow=True	Przetwarza wszystkie dostępne dane i kończy działanie
once=True	Przestarzały; zamiast niego używaj availableNow
continuous="1 second"	Eksperymentalne przetwarzanie o niskim opóźnieniu dla ograniczonego zestawu operacji

Dokumentacja: docs.databricks.com/en/structured-streaming/index.html

CZĘŚĆ 7

Wydajność

Liquid clustering

Liquid clustering to zalecany układ danych dla nowych tabel, który zastępuje zarówno partycjonowanie, jak i Z-ordering. Klucze klastrowania można zmienić bez przepisywania istniejących danych. Dla tabel Delta rozwiązanie jest dostępne ogólnie od Databricks Runtime 15.4 LTS.

```
CREATE TABLE main.silver.orders
CLUSTER BY (customer_id, order_date)
AS SELECT * FROM main.bronze.orders;

-- Cluster new and changed data; the operation is incremental
OPTIMIZE main.silver.orders;

-- Let Databricks choose and adapt the keys (Unity Catalog managed tables)
ALTER TABLE main.silver.orders CLUSTER BY AUTO;
```

Przy włączonej predictive optimization Databricks sam uruchamia **OPTIMIZE** na zarządzanych tabelach Unity Catalog.

OPTIMIZE i Z-ordering

Na tabelach bez liquid clustering **OPTIMIZE** kompaktuje małe pliki, a **ZORDER BY** grupuje powiązane wartości w tych samych plikach. Z-ordering pomaga najbardziej na kolumnach często filtrowanych i mających wiele różnych wartości.

```
OPTIMIZE main.silver.customers;
OPTIMIZE main.silver.transactions ZORDER BY (customer_id);
```

Photon

Photon to wektorowy silnik zapytań napisany w C++. Działa domyślnie w SQL warehouse i na zasobach serverless, a na klasycznych klastrach można go włączyć. Przyspiesza obciążenia SQL i DataFrame, takie jak skany, joiny i agregacje, bez zmian w kodzie.

Adaptive query execution

AQE jest domyślnie włączone. W trakcie wykonania zmienia plan zapytania: dobiera strategię joinów, łączy małe partycje shuffle i obsługuje przekos danych.

CZĘŚĆ 8

MLflow

MLflow śledzi eksperymenty uczenia maszynowego i jest wbudowany w każdy workspace Databricks.

```
import mlflow
from sklearn.ensemble import RandomForestClassifier

with mlflow.start_run(run_name="rf_baseline"):
    model = RandomForestClassifier(n_estimators=100, max_depth=5)
    model.fit(X_train, y_train)

    mlflow.log_param("n_estimators", 100)
    mlflow.log_param("max_depth", 5)
    mlflow.log_metric("accuracy", model.score(X_test, y_test))
    mlflow.sklearn.log_model(model, "model")
```

Uruchomienia są widoczne w sekcji **Experiments** na pasku bocznym workspace. MLflow obsługuje m.in. scikit-learn, XGBoost, PyTorch, TensorFlow, Hugging Face Transformers i LangChain.

Dokumentacja: docs.databricks.com/en/mlflow/index.html

CZĘŚĆ 9

Zaawansowana orkestracja

Lakeflow Jobs to znacznie więcej niż sekwencja notebooków.

Funkcja	Zastosowanie
Zależności zadań	Graf zadań, w tym zadań wykonywanych równolegle
Warunki if/else	Wybór kolejnego zadania na podstawie wartości zwróconej przez wcześniejsze zadanie
For each	Uruchomienie zadania dla każdego elementu listy
Task values	Przekazywanie niewielkich wyników między zadaniami przez <code>dbutils.jobs.taskValues</code>
Repair run	Ponowne uruchomienie tylko zadań zakończonych błędem i zadań od nich zależnych
Powiadomienia	E-mail, Slack, Microsoft Teams lub webhooki przy starcie, sukcesie albo błędzie

```
# Upstream task: publish a value
dbutils.jobs.taskValues.set(key="rows_processed", value=15000)

# Downstream task: read it
rows = dbutils.jobs.taskValues.get(taskKey="ingest", key="rows_processed", default=0)
```

CZĘŚĆ 10

Integracje

Narzędzie	Sposób połączenia
Power BI	Konektor Databricks do SQL warehouse albo Partner Connect
Tableau	Konektor Databricks albo ODBC i JDBC do SQL warehouse
Azure Data Factory	Połączona usługa (linked service) z aktywnością notebooka lub joba Databricks
dbt	Adapter dbt-databricks
Visual Studio Code	Rozszerzenie Databricks, które uruchamia kod na zasobach Databricks z poziomu IDE
Great Expectations	Biblioteka Pythona do kontroli jakości danych w notebookach

Materiały

Material	Adres
Dokumentacja Databricks	docs.databricks.com
Databricks Academy	databricks.com/learn/training
Forum społeczności	community.databricks.com
Blog Databricks	databricks.com/blog
Delta Lake na GitHubie	github.com/delta-io/delta